项目总结报告

日期：_____

组号	第八组	项目名称	InfoDigest (知食)
编程语言	Java (后端), JavaScript/TypeScript (前端), Python (爬虫)	开发平台和框架	Spring Boot, React, ECharts/React Flow, MySQL, Redis

项目工作小结

1. 项目实现了哪些功能？简要一一列出。

- **爬虫接入**：能够自动抓取 B站、知乎、小红书等第三方平台的内容。
- **大模型内容分析**：接入大模型，对文章内容进行提炼总结，生成我们项目特有的“五维营养成分表”。
- **知识图谱可视化**：根据用户的阅读历史生成个人的知识网络，前端可以支持节点的拖拽、缩放点击。
- **AI金句与用户画像**：AI智能提取文章中的金句，并且根据历史数据给用户画出一个AI用户画像。
- **记忆复习卡片**：系统会根据艾宾浩斯遗忘曲线，自动给用户生成知识点复习卡片。

2. 项目实现了哪些非功能？简要一一列出。

- **异步处理**：后端使用了基于 Redis 的异步队列来处理大模型 API 的调用，防止前端因为长时间等待大模型响应而卡死。
- **数据库设计灵活性**：在 MySQL 数据库里灵活运用了 JSON 字段，用来存各个平台抓回来的杂乱数据，不需要频繁去改表结构。
- **多端适配**：前端做了简单的响应式设计，PC端和手机端（Safari/Chrome浏览器）都能正常打开和看图谱。
- **前端性能优化**：知识图谱节点太多的时候，手机端做了层级按需渲染，防止内存爆炸导致浏览器崩溃卡顿。
- **自动化部署**：采用了 Docker 容器化把项目跑起来了，还简单配置了一下 CI/CD 流水线做自动发布。

3. 项目有哪些特点和亮点？例如业务特色、架构风格、设计模式、模型、算法、项目协同等

- **创意比较新**：提出了“信息营养学”的想法，用 AI 帮大家把平时手机上碎片化阅读的东西总结成有用的知识库。也帮助用户了解自己的信息获取质量和时长。
- **自我探索**：根据用户偏好的优质信息源和笔记，总结出“用户更容易被什么打动”，呈现为AI用户画像，帮助用户了解互联网中的自己是什么样子的。
- **前后端分离与解耦**：不仅前后端完全分离，还把常规的增删改查和特别耗时的大模型计算、图谱生成通过异步解耦了。
- **敏捷开发实践**：我们严格按照软工课上教的敏捷迭代开发，把大需求拆分到每个短冲刺里，并且努力做到了边写代码边写测试。

4. 项目的质量控制如何？简要一一列出。例如软件评审、单元测试、API 测试、系统的功能测试、性能测试、兼容性测试、界面测试等等

- **单元测试**：后端主力使用了 JUnit 和 Mockito 框架写单元测试，重点测了核心 Service 层的业务，覆盖率达到了 90% 以上。
- **接口 API 测试**：前后端联调前，用 Postman 手动调接口，检查后端的 JSON 格式和前端对不对得上。
- **系统与兼容性测试**：做了完整的系统测试，包括功能测试、性能测试、安全测试等。
- **文档与代码评审**：每次迭代我们都开了会，一起看用例图和类图，确保组内认知一致。

5.大模型工具辅助完成了哪些方面的软件开发实践活动?

- **画图和写文档**：帮我们把一些需求构思直接转成了规范的用例图、类图和 Mermaid 代码。
- **测试代码提效**：写单元测试的时候，大模型帮我们快速生成了 Mockito 的测试骨架，省了大量编废话代码的时间。
- **当高级搜索引擎**：写代码卡住的时候当搜索引擎用，比如帮忙写了优化图谱分类的提示词（Prompt），还有查了一些前端移动端适配的方法。
- **冲刺排期**：帮我们制定了最后的迭代冲刺计划表，还帮我们梳理了快验收时可能遇到的项目风险。

6. 大作业里，哪些代码开发、任务环节无法由大模型独立实现，需要人机协作或纯人工完成?

- **需求与功能设计的拍板**：到底要做个什么产品、具体长什么样子、核心保留哪些功能，大模型没法代替我们做主，需求方向和最终的产品设计必须由人工来决策。
- **前端视觉与交互测试**：很多前端 UI 界面的微调、手机上的真实交互体验（比如图谱拖拽手感好不好），AI 开发往往做不好，必须靠我们自己在真机上肉眼看、手动调。
- **环境与账号配置**：像云服务器的部署、真实数据库的搭建连接、还有第三方 API 密钥的申请等，这些基础的环境搭建和鉴权没法让 AI 代劳，只能人工一步步搞定。

项目组成员以及大模型工具对项目的贡献度（%）

姓名	需求	设计	编码	测试	项目管理*	小计
严戈	31%	25%	20%	17%	32%	**25%**
梅全全	17%	19%	34%	32%	23%	**25%**
周思哲	22%	35%	30%	15%	23%	**25%**
何左宜	30%	21%	16%	36%	22%	**25%**
合计	100%	100%	100%	100%	100%	**100%**

注：同学们应理解与评审大模型生成的内容，并负全部责任；项目管理包括项目计划、报告、沟通与协调等。

经验、教训和建议

经验 (Best Practice)：

1. **引入 Redis 消息队列**：用 Redis 替代了原来粗暴的数据库轮询来管理 AI 分析任务，不仅提升了性能，还彻底解决了定时扫描容易遗漏数据的问题。
2. **需求与设计先行**：在开始写代码前，老老实实写了详尽的《前景文档》，极大地统一了全队对核心功能的认知，提前做好的分工也让后期的并行开发非常顺利。
3. **果断重构**：开发过程中发现不合理、不合适的工具或者烂代码时，一定要敢于推翻重构，换掉之后往往能非常快速地解决卡脖子的问题。

教训 (Bad Practice)：

1. **低估了核心技术的壁垒**：前期做爬虫技术验证时只试了 B站，没意识到知乎、小红书等平台的反爬风控机制极其严苛，低估了抓取难度，导致后期到处碰壁。
2. **Git 代码冲突噩梦**：使用 Git 时没有早早合并分支，导致合代码时爆出满屏冲突。处理时稍有不慎就互相覆盖、丢失代码，出错后回溯重来相当折磨。
3. **单元测试欠债**：平时开发不写单元测试，全想着堆到最后补，结果发现时间根本来不及，只能疯狂熬夜补写测试用例。

项目组各成员签字：